

Protokoll zum Versuch
Aktivierung mit thermischen Neutronen
(Versuch 252)

Autor: Finn Zeumer 
Versuchspatnerin: Annika Künstle
Versuchsbegleiter: Lewin Dißelmeyer
Datum der Ausführung: 09.06.2026
Abgabedatum: 9. Juni 2026

Vorwort und Hinweise

Nutzung externer Tools

Für die sprachliche Ausarbeitung und Korrektur der Einleitung ([Kapitel 1.](#)) und der Durchführung ([Kapitel 3.](#)) wurden die KI *Lumo AI* sowie der *Duden Mentor* eingesetzt. Die inhaltliche Auswertung und die eigentliche Erstellung des Dokuments erfolgten jedoch vollständig ohne Unterstützung durch Künstliche Intelligenz, wenn nicht anders explizit gekennzeichnet. [[Dud26](#), [Pro26](#)]

Alle im Dokument enthaltenen Grafiken ohne Referenz wurden eigenständig in Inkscape erstellt bzw. nachbearbeitet oder sind in Python geplottete Grafiken. Der Python-Code sowie das LaTeX-Dokument wurden in der Entwicklungsumgebung Visual Studio Code (VS Code) angefertigt.

Eigener Code und Transparenz

Zur Durchführung der Auswertungen wurde eine eigene Python-Bibliothek entwickelt. Der gesamte Quellcode - sowohl die Python-Skripte als auch die LaTeX-Quellen - sind selbst erstellt und öffentlich online einsehbar.

Die gesamte Projekt-Struktur ist auf meinem GitHub zu finden. Hier ist sowohl der Python-Code, als auch der Latex-Code dokumentiert. [[Fin26a](#)]

Die benutzten Pythonfunktionen sind besonders im GitHub unter dem Namen „Papulator“ auffindbar. [[Fin26b](#)]

Verwendung universitärer Materialien

Dieses Dokument basiert auf dem Skript von Jens Wagner (Stand: 27. April 2026). Für die Einleitung wurden Grafiken aus diesem Skript übernommen. Diese Entnahmen sind transparent gekennzeichnet; die verwendeten Abbildungen stammen immer aus den Kapiteln des Skriptes, welche dieselbe Versuchsnummer und denselben Versuchsnamen wie im vorliegenden Dokument tragen. In diesem Versuch wurden die Grafiken aus *Versuch 252 Aktivierung mit thermischen Neutronen* verwendet. [[Wag26a](#)]

Inhaltsverzeichnis

Vorwort und Hinweise	i
1. Einleitung	1
1.1 Aufgabe und Motivation	1
1.2 Physikalische Grundlage	1
1.2.1 Erzeugung von Neutronenstrahlung	1
1.2.2 Abbremsen der Neutronen (thermische Neutronen)	1
1.2.3 Aktivierung von Silber (Erzeugung von Silberisotopen)	1
1.2.4 Aktivierung von Indium (Erzeugung von Indiumisotopen)	2
2. Messdaten	4
3. Durchführung	6
3.1 Messverfahren	6
3.1.1 Messung von Silber	6
3.1.2 Messung von Indium	6
3.2 Versuchsaufbau	6
4. Auswertung	7
4.1 Bestimmung der Halbwertszeit von Silber-Isotopen	8
4.1.1 Untergrundsstrahlung bei der Silbermessung	8
4.1.2 Bestimmung der Halbwertszeit	8
4.2 Bestimmung der Halbwertszeit von Indium-Isotopen	9
4.3 Bestimmung der Güte der Regressionen	11
5. Diskussion	12
5.1 Zusammenfassung	12
5.2 Analyse der Messwerte	12
5.3 Kritik	12
6. Python	13

1. Einleitung

1.1 Aufgabe und Motivation

Ziel dieses Versuchs ist die Bestimmung der Halbwertszeiten radioaktiver Isotope durch Aktivierung stabiler Kernspektren mittels thermischer Neutronen. Insbesondere sollen die Zerfallskonstanten und damit verbunden die Halbwertszeiten der Silberisotope ^{108}Ag und ^{110}Ag sowie des Indiumisotops ^{116}In bestimmt werden.

Die Untersuchung radioaktiven Zerfalls hat grundlegende Bedeutung für das Verständnis von Kernprozessen und ermöglicht Anwendungen in verschiedenen Bereichen der Physik, Medizin und Technik. Die Aktivierungsmethode stellt eine wichtige Technik dar, um stabile Isotope in radioaktive Quellen zu überführen, deren Eigenschaften dann präzise vermessen werden können. Durch den Vergleich der experimentell bestimmten Werte mit Literaturdaten lässt sich zudem die Genauigkeit der Messverfahren überprüfen und systematische Fehler identifizieren.

1.2 Physikalische Grundlage

1.2.1 Erzeugung von Neutronenstrahlung

Eine radioaktive Quelle kann durch Aktivierung stabiler Isotope mithilfe von Kernreaktionen hergestellt werden. Für die Anregung solcher Kernreaktionen werden häufig Neutronen verwendet, da diese elektrisch neutral sind und somit nicht der Coulomb-Wechselwirkung unterliegen. Der entsprechende Atomkern kann ein Neutron daher besonders leicht einfangen.

Für diesen Versuch wird eine Neutronenquelle eingesetzt, die aus einem Gemisch von Berylliumspänen und dem α -Strahler Americium-241 (^{241}Am) besteht. Es tritt folgende Kernreaktion auf:



Bei dem Beschuss von Beryllium mit α -Teilchen entsteht Kohlenstoff sowie ein freies Neutron. Das so erzeugte Neutron besitzt eine hohe kinetische Energie im Bereich von etwa 1 bis 10 MeV.

1.2.2 Abbremsen der Neutronen (thermische Neutronen)

Die bei der oben beschriebenen Reaktion entstehenden schnellen Neutronen müssen zunächst abgebremst werden, damit sie für bestimmte Kernreaktionen nutzbar gemacht werden können. Je langsamer ein Neutron ist, desto höher ist die Wahrscheinlichkeit, dass es von einem Atomkern eingefangen wird. Der Abbremsvorgang erfolgt hier durch einen Paraffinblock, der die Neutronenquelle umgibt.

Paraffin ist für diesen Prozess besonders geeignet, da es zahlreiche Wasserstoffkerne (Protonen) enthält. Trifft ein Neutron auf ein Proton, kommt es zu einem elastischen Stoß, bei dem das Neutron etwa die Hälfte seiner kinetischen Energie verliert. Bei Stößen mit Kohlenstoffkernen im Paraffin gehen dagegen weniger Energie verloren, da die Kohlenstoffkerne deutlich größer als die Neutronen sind. Die nach diesem Prozess abgebremsten Neutronen besitzen eine nahezu thermische Energie und werden als thermische Neutronen bezeichnet.

1.2.3 Aktivierung von Silber (Erzeugung von Silberisotopen)

Natürliches Silber setzt sich zusammen aus 51 % ^{107}Ag und 49 % ^{109}Ag . Wird natürliches Silber wie oben erläutert aktiviert, entsteht aus ^{107}Ag das Isotop ^{108}Ag und aus ^{109}Ag das Isotop ^{110}Ag . Beide entstandenen Isotope stellen β -Strahler dar, die durch Zerfall in ^{108}Cd bzw. ^{110}Cd übergehen.

Die Halbwertszeiten dieser beiden Silberisotope unterscheiden sich um etwa einen Faktor sechs. ^{110}Ag weist eine kürzere Halbwertszeit auf und erreicht daher den Sättigungswert vor ^{108}Ag . Bei längerer Aktivierungszeit dominiert ^{108}Ag aufgrund seiner deutlich längeren Halbwertszeit. Damit wird für kurze Aktivierungszeiten hauptsächlich ^{110}Ag gebildet. Das Aktivitätsverhältnis der beiden Isotope verändert sich also mit der Bestrahlungszeit während der Aktivierung.

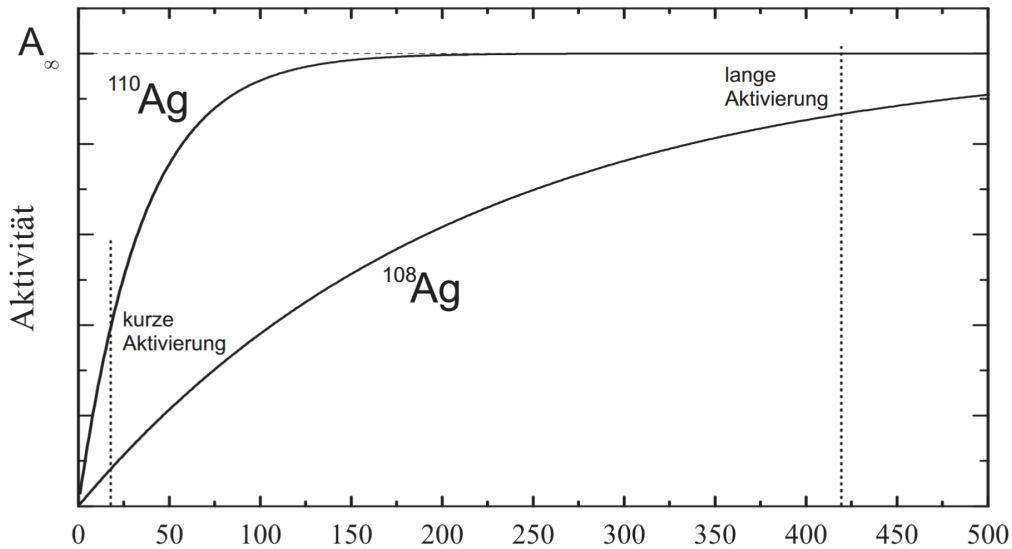


Abbildung 1.I:

Aktivierung der natürlichvorkommenden Silberisotope 51 % ^{107}Ag und 49 % ^{109}Ag . [Wag26b]

Während des Aktivierungsprozesses werden kontinuierlich neue radioaktive Kerne gebildet, während gleichzeitig bereits vorhandene zerfallen. Die Aktivität $A(t)$ (Zahl der Zerfälle pro Sekunde) folgt dem Gesetz:

$$A(t) = A_{\infty} (1 - e^{-\lambda t}) \quad (1.2)$$

Dabei bezeichnet A_{∞} die maximale Aktivität im Gleichgewicht (Sättigungsaktivität) und λ beschreibt die Zerfallskonstante des Isotops. Für $t \rightarrow \infty$ nähert sich $A(t)$ asymptotisch an A_{∞} an.

1.2.4 Aktivierung von Indium (Erzeugung von Indiumisotopen)

Um stabiles Indium zu aktivieren, wird es analog bestreut. Durch die Bestrahlung kommen zur Bildung zweier Isomere eines radioaktiven Indiumisotops. Zum einen entsteht das Isotop ^{116}In , welches sich im energetischen Grundzustand befindet, zum anderen $^{115\text{m}}\text{In}$, welches sich im metastabilen Zustand befindet. Beide Isotope sind β -Strahler, zeigen aber unterschiedliche Halbwertszeiten. Beide Isotope des Indiums zerfallen zu ^{116}Sn (Zinn).

Nach Abschluss der Aktivierung beginnt der rein radioaktive Zerfall, da keine neuen radioaktiven Kerne mehr erzeugt werden. Dann folgt:

$$A(t) = A_0 \cdot e^{-\lambda t} \quad (1.3)$$

Für die Halbwertszeit gilt somit:

$$T_{1/2} = \frac{\ln(2)}{\lambda} \quad (1.4)$$

Zusätzlich kann auch die mittlere Lebensdauer τ definiert werden als:

$$\tau = \frac{1}{\lambda} \quad (1.5)$$

Durch Messung der Zerfallsrate über die Zeit lässt sich die Zerfallskonstante bestimmen, woraus sich gemäß Gleichung 1.4 die Halbwertszeit berechnen lässt. Dies bildet die Grundlage der experimentellen Auswertung.

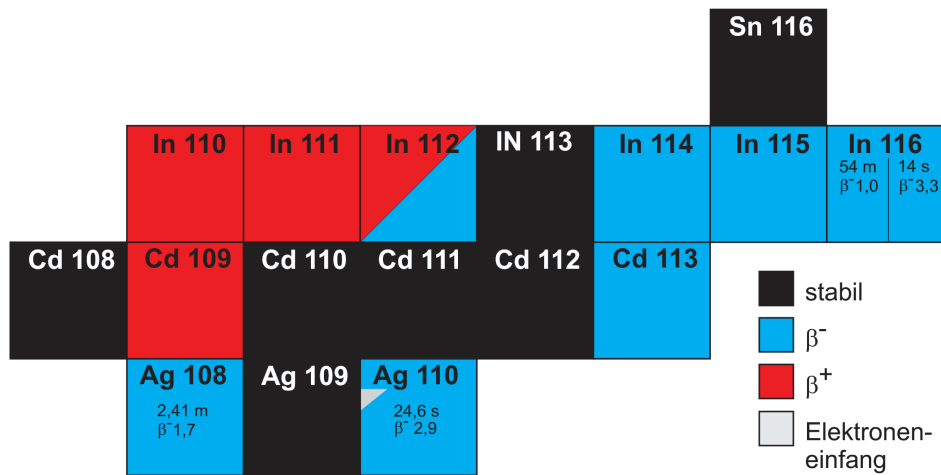


Abbildung 1.II:

Ausschnitt der Nuklidkarte mit den Halbwertszeiten und der Energie der emittierten Strahlung[Wag26b]

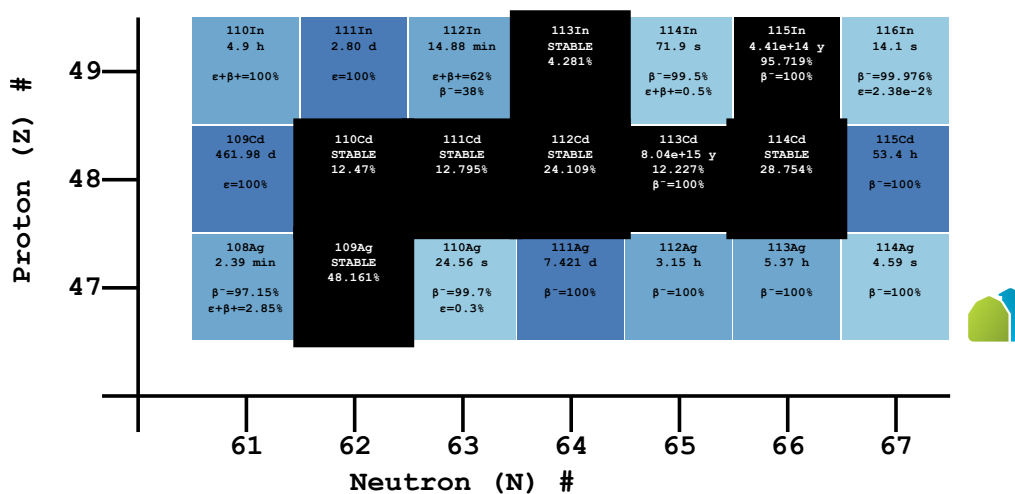


Abbildung 1.III:

Ausschnitt Nuklidkarte nach NuDat (<https://www.nndc.bnl.gov/nudat3/>)

Messprotokoll zum Versuch „Aktivierung mit thermische Neutronen“ (252)

Experimentator*innen: Annika Künste, Finn Zeumer

Datum der Durchführung: 08.06.2026

Spannung des Zählrohrs: $U_z = \underline{(500 \pm 10) V}$

Es wird eine Untergrundmessung (ohne Präparat) durchgeführt.

- Totzeit: 10s
- Messzeit: $\sim 500s$

Aufgabe 1)

Es wird die Halbwertszeit von Silber bestimmt.

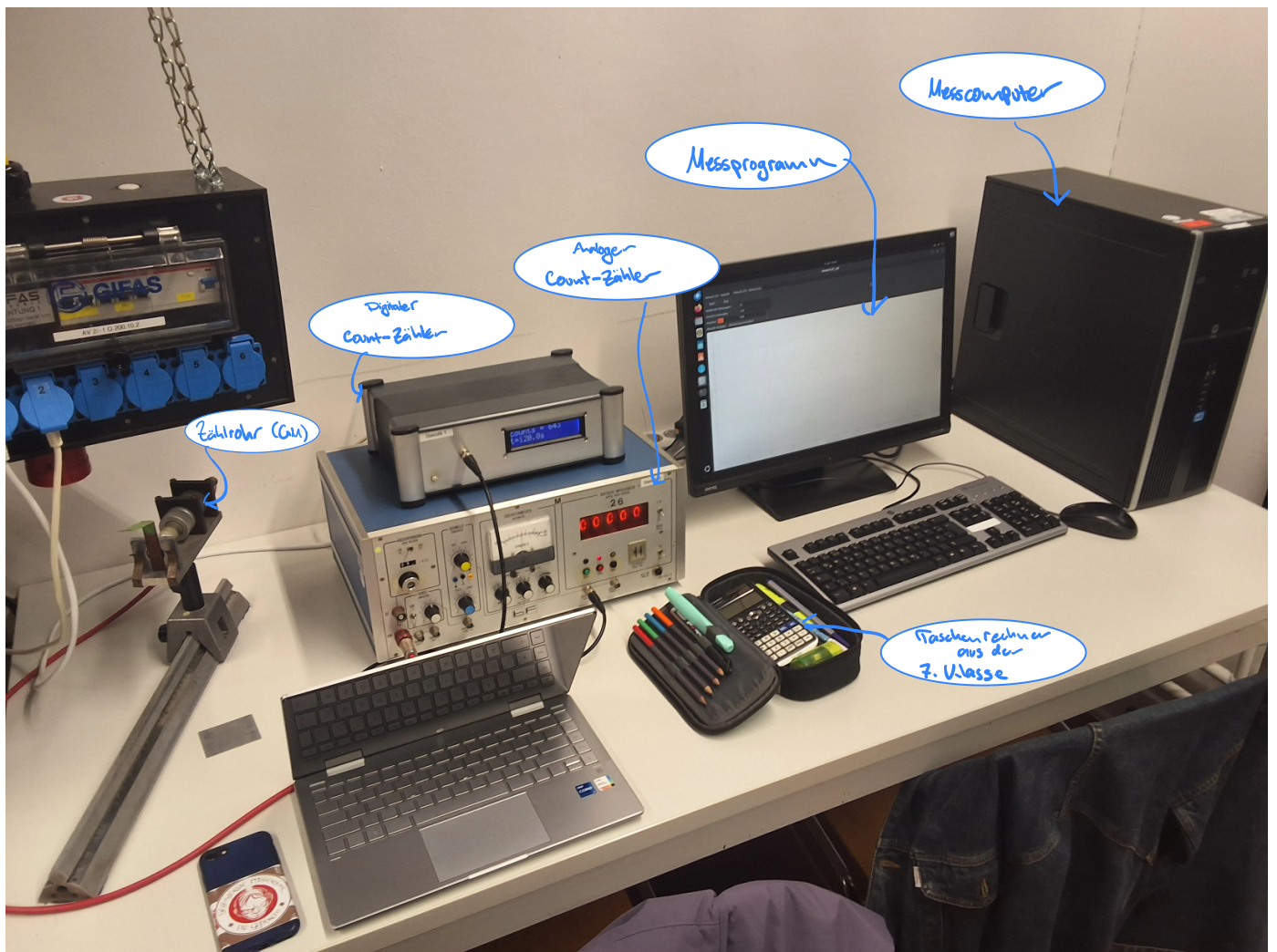
- Totzeit: 10s
- Aktivierungszeit 7 Minuten
- Messzeit: 400s

Die Messung soll drei Mal durchgeführt werden.

Aufgabe 2)

Es wird die Halbwertszeit von Indium bestimmt.

- Totzeit: 120s
- Messzeit: 50 min $\hat{=}$ 3000s
- Aktivierungszeit: 4



Experimentatoren

26.26

L.D



3. Durchführung

3.1 Messverfahren

3.1.1 Messung von Silber

Für die Messung der Halbwertszeit von aktiviertem Silber wird eine Geiger-Müller-Zählrohrspannung zwischen 500 V und 550 V am Betriebsgerät eingestellt. Zunächst wird die Untergrundzählrate bestimmt, um diese später von den Messwerten abziehen zu können. Dazu wird die Torzeit auf 10 s eingestellt und 50 Messungen durchgeführt, sodass die Untergrundzählrate über einen Zeitraum von etwa acht Minuten erfasst wird.

Anschließend wird die eigentliche Silbermessung durchgeführt. Die Träger mit den Silberblechen werden zur Aktivierung für sieben Minuten in der Neutronenquelle platziert. Nach Beendigung der Aktivierung wird das Präparat schnellstmöglich an das Zählrohr gebracht und dort mit einem Aluminiumblech fixiert. Die Messung startet mit einer Torzeit von 10 s über eine Gesamtzeit von 400 s. Dieser Zyklus wird insgesamt viermal wiederholt, um die statistische Unsicherheit zu reduzieren. Die vier Messreihen werden anschließend addiert und gemeinsam ausgewertet.

3.1.2 Messung von Indium

Für die Messung der Halbwertszeit von aktiviertem Indium wird analog vorgegangen. Das Messintervall wird dabei auf 120 s eingestellt. Das aktivierte Indium-Präparat wird in der Halterung fixiert und über einen Zeitraum von etwa 50 Minuten gemessen. Auch hier wird zuvor die Untergrundzählrate bestimmt, wobei die Torzeit entsprechend angepasst wird. Da bei der Aktivierung von Indium neben dem Grundzustand auch ein metastabiler Zustand mit sehr kurzer Halbwertszeit gebildet wird, wird der erste Messpunkt für den Fit vernachlässigt, da dieser vom eigentlichen Zerfall der betrachteten Isotope abweicht.

3.2 Versuchsaufbau

Der Versuchsaufbau besteht aus mehreren Komponenten. Eine Neutronenquelle, bestehend aus einem Gemisch von Berylliumspänen und dem α -Strahler Americium-241, wird von einem Paraffinblock umgeben, der die schnellen Neutronen auf thermische Energien abbremst. Die zu aktivierenden Proben (Silber- und Indiumbleche) werden in einer Probenhalterung positioniert und während der Aktivierungsphase in die Nähe der Neutronenquelle gebracht.

Nach der Aktivierung wird das Präparat in die Position eines Geiger-Müller-Zählrohrs überführt, das mit einem Betriebsgerät verbunden ist. Der Impulszähler zählt die registrierten Zerfälle über jeweils festgelegte Zeitintervalle (Torzeiten). Die Daten werden auf einem PC gesammelt und anschließend mit geeigneter Software ausgewertet.

4. Auswertung

Fehlerrechnung

Für die statistische Auswertung von n Messwerten x_i werden folgende Größen definiert [Wag25]:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad \text{Arithmetisches Mittel} \quad (4.1)$$

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad \text{Varianz} \quad (4.2)$$

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2} \quad \text{Standardabweichung} \quad (4.3)$$

$$\Delta\bar{x} = \frac{\sigma}{\sqrt{n}} = \sqrt{\frac{1}{n(n-1)} \sum_{i=1}^n (\bar{x} - x_i)^2} \quad \text{Fehler des Mittelwerts} \quad (4.4)$$

$$\Delta f = \sqrt{\left(\frac{\partial f}{\partial x} \Delta x\right)^2 + \left(\frac{\partial f}{\partial y} \Delta y\right)^2} \quad \text{Gauß'sches Fehlerfortpflanzungsgesetz für } f(x, y) \quad (4.5)$$

$$\Delta f = \sqrt{(\Delta x)^2 + (\Delta y)^2} \quad \text{Fehler für } f = x + y \quad (4.6)$$

$$\Delta f = |a| \Delta x \quad \text{Fehler für } f = ax \quad (4.7)$$

$$\frac{\Delta f}{|f|} = \sqrt{\left(\frac{\Delta x}{x}\right)^2 + \left(\frac{\Delta y}{y}\right)^2} \quad \text{relativer Fehler für } f = xy \text{ oder } f = x/y \quad (4.8)$$

$$\sigma = \frac{|a_{lit} - a_{gem}|}{\sqrt{\Delta a_{lit}^2 + \Delta a_{gem}^2}} \quad \text{Berechnung der signifikanten Abweichung} \quad (4.9)$$

4.1 Bestimmung der Halbwertszeit von Silber-Isotopen

4.1.1 Untergrundstrahlung bei der Silbermessung

In dieser Sektion soll der Zerfall des Silbers betrachtet werden. Dafür wurde zunächst ohne Probe die Untergrundstrahlung bestimmt. Dazu wurde für die Torzeit 10s und für die Messzeit 500s eingestellt. Es wird das [arithmetische Mittel \(4.1\)](#) und seine Ungenauigkeit über den [Fehler des Mittelwertes \(4.4\)](#) bestimmt. Zusätzlich wird die Messung mit einem Faktor n multipliziert, wobei n die Anzahl der Messreihen ist. Für diese Silbermessung gilt $n = 4$

$$\overline{N_{\text{UG,Ag}}} = (13.0 \pm 1.1)s^{-1} \quad (4.10)$$

4.1.2 Bestimmung der Halbwertszeit

Die vier Messreihen werden aufsummiert und eine Gesamtmessreihe zu erhalten. Diese wird gegend die Zeit geplottet. Durch die Messdaten wird eine Regression der Form

$$f(x) = A_1 e^{-\lambda_1 x} + A_2 e^{-\lambda_2 x} + y_0. \quad (4.11)$$

gelegt. Dass es eine Summation aus zwei Exponentialfunktionen entspricht liegt daran, dass zwei Isotope von Silber natürlich vorkommen. Der Index 1 soll das Isotop ^{110}Ag beschreiben und der Index 2 das Isotop ^{108}Ag . Der Term y_0 beschreibt die Untergrundstrahlung. Die Messdaten mit der Regression sind der [Abbildung 4.I](#) zu entnehmen.

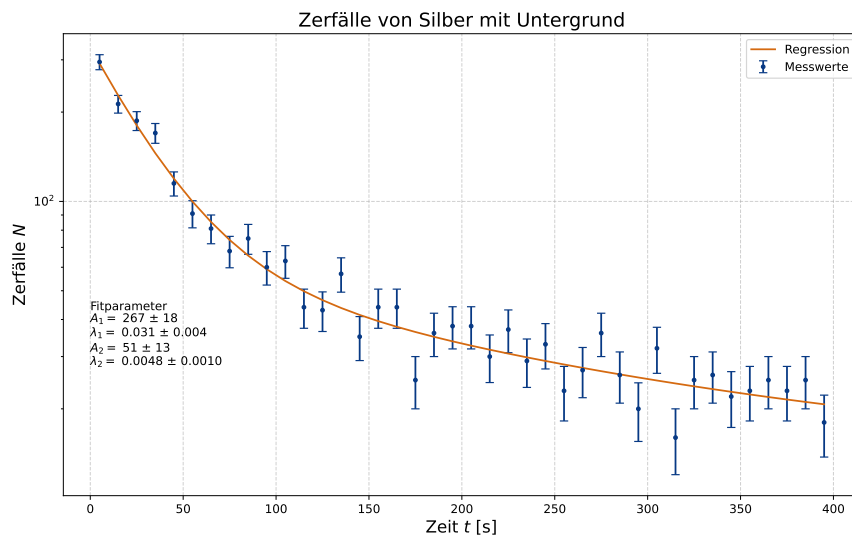
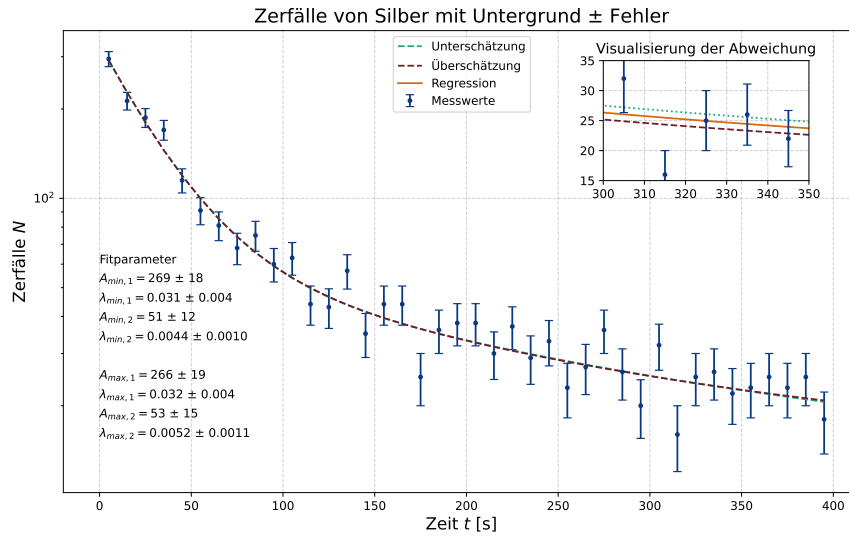


Abbildung 4.I:
Bestimmte Regressionen für die Silbermessung.

Es werden zwei weitere Fits erstellt, welche dazu dienen, eine präzisere Ungenauigkeit der Untergrundmessung zu bestimmen. Dazu wird der Untergrund über- und unterschätzt, so ergeben sich ein Maximal- und ein Minimalwert. Aus denen wird das [arithmetische Mittel \(4.1\)](#) gebildet, dies entspricht der Ungenauigkeit der gemessenen Untergrundstrahlung. Es gilt also die Differenzen $\sigma^- = |\lambda - \lambda_{\min}|$ und $\sigma^+ = |\lambda + \lambda_{\max}|$ zu bestimmen. Aus diesen wird das [arithmetische Mittel \(4.1\)](#) gebildet. Nach [Gleichung 4.6](#) können nun die Ungenauigkeit der Untergrundmessung und dem [Fehler des Mittelwertes \(4.4\)](#) verrechnet werden. Dies ist der Gesamtfehler der Zerfallskonstanten $\Delta\lambda$.

Die Fitparameter beider Regressionen und die Regression selbst sind der [Abbildung 4.II](#) zu entnehmen.

Aus der Zerfallskonstanten lassen sich die Halbwertszeit $T_{1/2}$ nach [Gleichung 1.4](#) und die Lebensdauer τ nach [Gleichung 1.5](#) bestimmen. Die Ergebnisse sind der [Tabelle 4.I](#) zu entnehmen.

**Abbildung 4.II:**

Bestimmte Regressionen mit Unter- und Überschätzung für die Silbermessung.

Tabelle 4.I:

Messergebnisse der Silbermessung.

Größe	$\overline{N_{UG,Ag}} [s^{-1}]$	$\sigma^- [s^{-1}]$	$\sigma^+ [s^{-1}]$	Gesamt
$\lambda_1 \cdot 10^{-2} [s^{-1}]$	3.1 ± 0.4	0.027	0.032	3.15 ± 0.03
$\lambda_2 \cdot 10^{-3} [s^{-1}]$	4.8 ± 1.0	0.35	0.40	4.8 ± 0.4
$T_{1/2}^1 [s]$				22.04 ± 0.21
$T_{1/2}^2 [s]$				145 ± 11
$\tau_1 [s]$				31.8 ± 0.3
$\tau_2 [s]$				209 ± 16

4.2 Bestimmung der Halbwertszeit von Indium-Isotopen

Analog zur Auswertung des Silbers in der [Sektion 4.1](#) soll hier die Indiumprobe untersucht werden. Hier ist nur ein Isotop relevant, denn ^{116m}In hat eine Halbwertszeit von $T_{1/2} = 14s$, wohingegen ^{116}In eine Halbwertszeit von $T_{1/2} = 54min$ hat. Da die Torzeit auf 10s gestellt ist, muss der erste Messwert herausgenommen werden, da dieser den Zerfall von ^{116}In im großen Maße mit gemessen hat. Die Regression für die Indiummessung ergibt nach [Gleichung 1.3](#) sich zu

$$f(x) = A_0 e^{-\lambda_1 x} + y_0. \quad (4.12)$$

Dabei sind die Variablen wie in [Gleichung 4.11](#) zu interpretieren. Auch hier muss der Untergrund betrachtet werden. Da dieses Mal die Torzeit auf 120s gestellt wurde, muss das [arithmetische Mittel \(4.1\)](#) der Untergrundmessung mit 12 multipliziert werden. Wichtig zu betonen ist, dass zwei Versuchsaufbauten parallel betrieben wurden. Es ergibt sich:

$$\overline{N_{UG,In}} = (35.52 \pm 0.25) s^{-1} \quad (4.13)$$

Die Messdaten, sowie die Regression sind der [Abbildung 4.III](#) zu entnehmen.

Auch hier soll der Untergrund genauer bestimmt werden, es werden wieder eine Maximal- und Minimalabschätzung unternommen. Die Regressionen und deren Fitparameter sind der [Abbildung 4.IV](#) zu entnehmen.

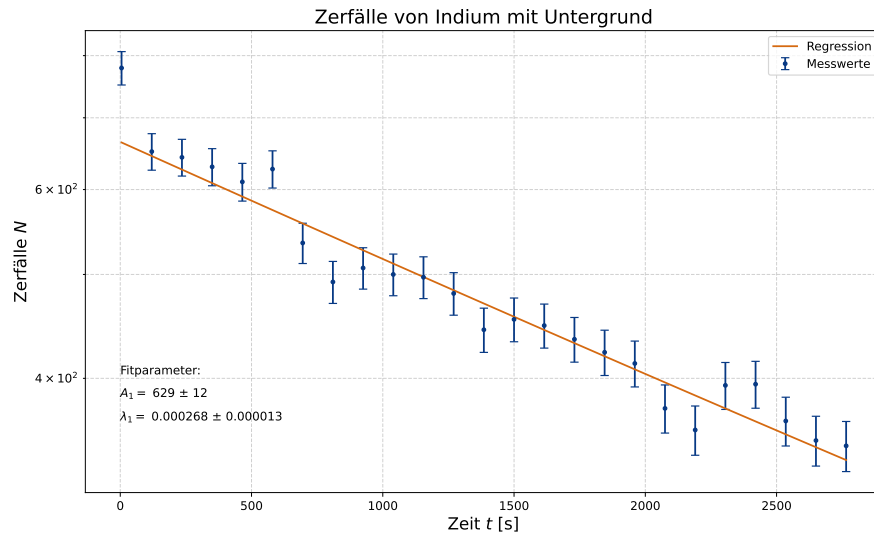


Abbildung 4.III:
Bestimmte Regressionen für die Indiummessung.

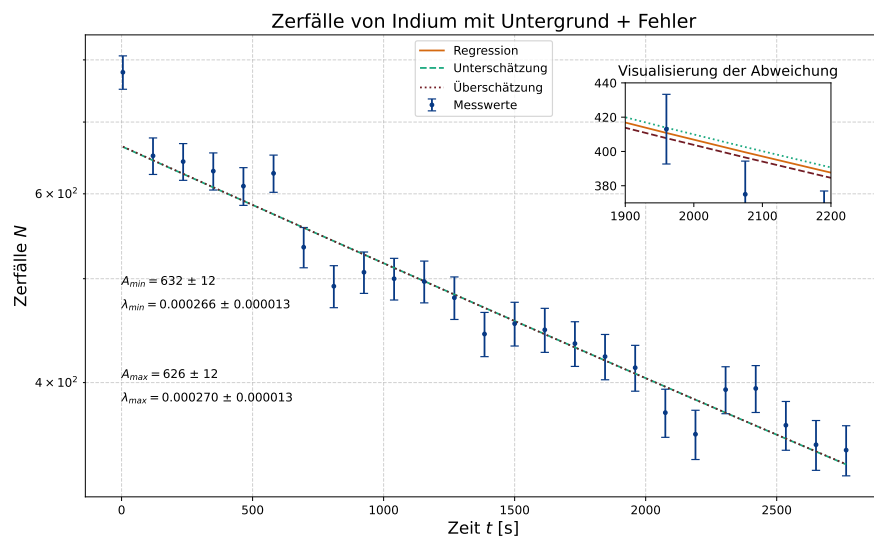


Abbildung 4.IV:
Bestimmte Regressionen mit Unter- und Überschätzung für die Indiummessung.

Tabelle 4.II:
Messergebnisse der Indiummessung.

Größe	$\overline{N}_{UG,In}$	$\sigma^- [s^{-1}]$	$\sigma^+ [s^{-1}]$	Gesamt
$\lambda \cdot 10^{-4} [s^{-1}]$	(2.68 ± 0.13)	0.0190	0.0193	2.677 ± 0.019
$T_{1/2} [s]$				2589 ± 19
$\tau [s]$				3730 ± 30

4.3 Bestimmung der Güte der Regressionen

Zuletzt sollen die bestimmten Regressionen auf ihre Güte geprüft werden. Dazu wird zu nächst das Chi-Quadrat (χ^2) bestimmt. Zudem wird dieses via Division mit den Freiheitsgraden dividert. Ist das $\chi^2_{\text{red}} \approx 1$ ¹, ist der Fit als sinnvoll zu erachten. Die Freiheitsgrade (DoF) können via

$$\text{DoF} = n_{\text{mess}} - n_{\text{par}} \quad (4.14)$$

bestimmt werden. Mit der Anzahl der Messungen n_{mess} und der Anzahl der freien Parameter n_{par} . Zuletzt kann die Güte einer Regression noch über die Wahrscheinlichkeit eines Fits bestimmt werden. Diese Methode nimmt die bestimmten Regressionsparameter als wahr an und verteilt die Messwerte normal um den Mittelwert. Die Übereinstimmung der theoretischen und experimentellen Werte wird dann bestimmt. Die bestimmten Werte sind der [Tabelle 4.III](#) zu entnehmen.

Tabelle 4.III:
Ergebnisse der Güte der Regressionen für Indium und Silber.

Größe	Silber	Indium
χ^2	36.71	39.37
χ^2_{red}	1.02	1.71
Wahrscheinlichkeit[%]	44.0	2.0%

¹Ein Wert von = 1 wäre ein optimaler Fit.

5. Diskussion

5.1 Zusammenfassung

Hallo, heute haben wir den Versuch 252 ausgeführt. Es hat ganz viel Spaß gemacht. Besonders toll fand ich die lange Wartezeit, da ich ein sehr geduldiger Mensch bin. Die Ergebnisse sind eigentlich auch ganz dolle geworden, also beschweren kann man sich da normalerweise eher weniger.

5.2 Analyse der Messwerte

Tabelle 5.I:

Messergebnisse der Indiummessung mit [Standardabweichung \(4.3\)](#) zum Literaturwert nach [Abbildung 1.II](#).

Isotrop	$T_{1/2}[s]$	Literaturwert [s]	Relativer Fehler [%]	Std.Abw. σ
^{108}Ag	145 ± 1111	144.6	7.8	0.01
^{110}Ag	22.04 ± 0.21	24.6	0.93	12.48
^{116}In	2589 ± 19	3240	0.71	35.13

5.3 Kritik

6. Python

Der gesamte Pythoncode ist auf auf meinem GitHub unter <https://github.com/FinnZeumer/PAP-2> [Fin26a] zu finden. Zudem ist hier auch der Sorce-Code für dieses Projekt selbst, falls Interesse besteht diesen zu sehen.

Der Code ist primär selbst geschrieben. Es wird jedoch die in Teilen die eingebaute Funktion des „MS Copilot“ in Visual Studio Code verwendet. Wenn es nicht anders gekennzeichnet wird, wird diese jedoch als Unterstützung genutzt und nicht zum generieren ganzer Methoden. Zudem haben wir einen Ordner mit unseren Code-Projekten in der Freundesgruppe. Hier teilen wir alle unseren Code.

PAP 2.2 – Python-Auswertung

Wichtige Funktionen für das PAP.

Alle in diesem Dokument geschriebenen Zeilen Code sind von mir eigenständig geschrieben. Für ein sauberes und strukturiertes Dokument wurde eine eigene Pythonlibrary geschrieben, welche insbesondere ans PAP angepasst wurde. Dieses ist unter dem Namen "Papulator" auf meinem GitHub zu finden. Diese soll stetig ausgearbeitet werden. Der Stand dieses Protokolls ist der 29.04.2026.

Inhaltsverzeichnis zur Auswertung

- [Definition der Versuchsvariablen](#)
- [Import aller genutzen Libraries](#)
- [Auswertung der Aufgaben](#)
 - [Aufgabe 1](#)
 - [Aufgabe 2](#)

Definition der Versuchsvariablen

Im Folgenden sind Versuchsvariablen definiert, die für einen besseren Workflow sorgen sollen. Diese werden zum exportieren und überschreiben von Dateien wichtig sein und ermöglichen es, diese Datei für jeden Versuch zu benutzen und lediglich die Variablen zu verändern. Es ist jedoch empfohlen eine Kopie der Forlage für jeden Versuch zu machen, damit dieses Dokument strukturiert bleibt.

Zudem sind wichtige Konstanten definiert, die immer wieder auftauchen.

```
In [1]: versuchsnummer = "252"
versuchsname = "Aktivierung_mit_hermischen_Neutronen"
aufgabe = "02"
plt_save_folder = f"./img/plots/"
```

Import aller genutzen Libraries

```
In [2]: # Eigene Pythonlibrary für das Pap
import Papulator as pap
from Papulator import Colors as c
from Papulator import const
from Papulator import Sympy_Symbols as sym

# Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
from scipy.optimize import curve_fit
from scipy.stats import norm
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial
from scipy.integrate import quad

import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

import astropy.units as u
```

```
# Zum Auslesen von Dateien und ähnlichem
import os
import os.path

import pandas as pd
import csv
import re

from io import StringIO

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars
```

Auswertung der Aufgaben

Aufgabe 1) Silber

```
In [3]: # Untergrundmessung
unterg = np.loadtxt('./data/ag/untergrund_ag.txt', usecols = [1], skiprows=4)

print(np.mean(unterg)*12)
print(np.std(unterg) / np.sqrt(len(unterg)))
print('- '*15)

mean_unterg = 4 * np.mean(unterg)
sig_mean_unterg = np.std(4 * unterg) / np.sqrt(len(unterg))

print(fr'Der Mittelwert der Untergrundmessung liegt bei: {pap.round_sig_digs(sig_mean_unterg,mean_unterg)[2]}')
```

38.88
0.2822197725178021

Der Mittelwert der Untergrundmessung liegt bei: 13.0 \pm 1.1

```
In [4]: # Definition der Regressionsfunktion
y0 = mean_unterg
def expo(x, A1, l1, A2, l2):
    return A1 * exp(-l1 * x) + A2 * exp(-l2 * x) + y0
```

```
In [5]: aufgabe = '1a'

# Import der Messdaten
n1 = np.loadtxt('data/ag/silber1.txt', usecols = [1], skiprows=4)
n2 = np.loadtxt('data/ag/silber2.txt', usecols = [1], skiprows=4)
n3 = np.loadtxt('data/ag/silber3.txt', usecols = [1], skiprows=4)
n4 = np.loadtxt('data/ag/silber4.txt', usecols = [1], skiprows=4)

N = n1 + n2 + n3 + n4
sig_N = np.sqrt(N)
t = np.arange(5, 405, 10)

# Regression
popt, pcov = curve_fit(expo, t, N, sigma = sig_N, p0 = [250, 0.2, 50, 0.001])

# Fitparameter
A_1 = pop[0]
sig_A_1 = pcov[0,0]
err_A_1 = sqrt(sig_A_1)

l_1 = pop[1]
sig_l_1 = pcov[1,1]
err_l_1 = sqrt(sig_l_1)

A_2 = pop[2]
sig_A_2 = pcov[2,2]
err_A_2 = sqrt(sig_A_2)

l_2 = pop[3]
sig_l_2 = pcov[3,3]
err_l_2 = sqrt(sig_l_2)

# Plot
plt.figure(figsize = (12,7))

# Messungen
plt.errorbar(
```

```

t,
N,
yerr = sig_N,
fmt = '.',
capsize = 3.5,
label = 'Messwerte',
color = c.BLUE
)

plt.xlabel('Zeit t[s]', size = 14)
plt.ylabel('Zerfälle N$', size = 14)
plt.title('Zerfälle von Silber mit Untergrund', size = 16)
plt.yscale('log')

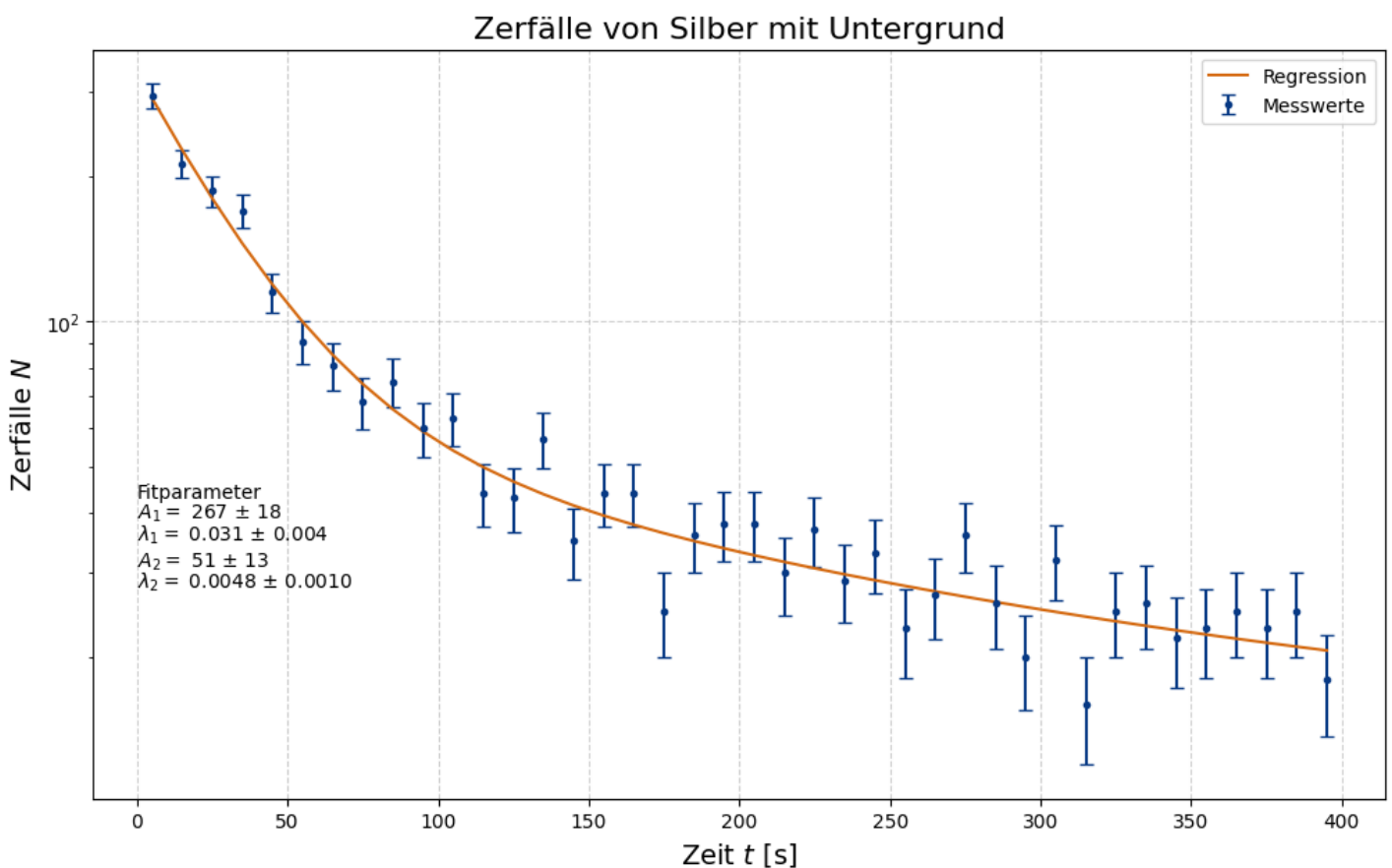
# Fit
plt.plot(
    t,
    expo(t, *popt),
    label = 'Regression',
    color = c.ORANGE
)

plt.text(0, 4.3e1, 'Fitparameter')
plt.text(0, 3.9e1, f'$A_1 = $ {pap.round_sig_digs(err_A_1, A_1)[3]}')
plt.text(0, 3.5e1, f'$\\lambda_1 = $ {pap.round_sig_digs(err_l_1, l_1)[3]}')
plt.text(0, 3.1e1, f'$A_2 = $ {pap.round_sig_digs(err_A_2, A_2)[3]}')
plt.text(0, 2.8e1, f'$\\lambda_2 = $ {pap.round_sig_digs(err_l_2, l_2)[3]}')

plt.grid(True, linestyle='--', alpha=0.6)

plt.legend(loc = 'best')
plt.savefig(f'{plt_save_folder}Regression_der_Messdaten_{aufgabe}.pdf', dpi=300)
plt.show()

```



```

In [6]: # Güte des Fits
chi2_ = np.sum((expo(t, *popt) - N)**2 / sig_N**2)
dof = len(N) - 4 #dof:degrees of freedom, Freiheitsgrad
chi2_red = chi2_/dof

print("chi2 =", round(chi2_,2))
print("chi2_red =", round(chi2_red,2))

prob = np.round(1 - chi2.cdf(chi2_,dof),2) * 100
print("Wahrscheinlichkeit =", prob, "%")

chi2 = 36.71
chi2_red = 1.02
Wahrscheinlichkeit = 44.0 %

```

```

In [7]: aufgabe = '1b'

```

```

# Fit unter Berücksichtigung des Untergrund-Fehlers
# Plot
plt.figure(figsize = (12,7))

# Messungen
plt.errorbar(
    t,
    N,
    yerr = sig_N,
    fmt = '.',
    capsize = 3.5,
    label = 'Messwerte',
    color = c.BLUE
)

plt.xlabel('Zeit  $t$  [s]', size = 14)
plt.ylabel('Zerfälle  $N$ ', size = 14)
plt.title('Zerfälle von Silber mit Untergrund  $\lambda$  Fehler', size = 16)
plt.yscale('log')

# Fit
y0 = mean_unterg - sig_mean_unterg # Unterschätzt
popt_min, pcov_min = curve_fit(expo, t, N, sigma = sig_N,
p0 = [250, 0.2, 50, 0.01])

# Fitparameter
A_min_1 = popt_min[0]
sig_A_min_1 = pcov_min[0,0]
err_A_min_1 = sqrt(sig_A_min_1)

l_min_1 = popt_min[1]
sig_l_min_1 = pcov_min[1,1]
err_l_min_1 = sqrt(sig_l_min_1)

A_min_2 = popt_min[2]
sig_A_min_2 = pcov_min[2,2]
err_A_min_2 = sqrt(sig_A_min_2)

l_min_2 = popt_min[3]
sig_l_min_2 = pcov_min[3,3]
err_l_min_2 = sqrt(sig_l_min_2)

plt.plot(
    t,
    expo(t, *popt_min),
    label = 'Unterschätzung',
    ls = '--',
    color = c.LIGHT_GREEN
)

# Fitparameter als Text im Plot
text_pos_upper = 4.1
plt.text(0, exp(text_pos_upper),
'Fitparameter')

plt.text(0, exp(text_pos_upper - 0.15),
'$A_{min,1} = $' + str(pap.round_sig_digs(err_A_min_1, A_min_1)[3]))

plt.text(0, exp(text_pos_upper - 0.3),
'$\lambda_{min,1} = $' + str(pap.round_sig_digs(err_l_min_1, l_min_1)[3]))

plt.text(0, exp(text_pos_upper - 0.45),
'$A_{min,2} = $' + str(pap.round_sig_digs(err_A_min_2, A_min_2)[3]))

plt.text(0, exp(text_pos_upper - 0.6),
'$\lambda_{min,2} = $' + str(pap.round_sig_digs(err_l_min_2, l_min_2)[3]))

y0 = mean_unterg + sig_mean_unterg # Überschätzt
popt_max, pcov_max = curve_fit(expo, t, N, sigma = sig_N, p0 = [250, 0.2, 50, 0.01])

# Fitparameter
A_max_1 = popt_max[0]
sig_A_max_1 = pcov_max[0,0]
err_A_max_1 = sqrt(sig_A_max_1)

l_max_1 = popt_max[1]
sig_l_max_1 = pcov_max[1,1]
err_l_max_1 = sqrt(sig_l_max_1)

A_max_2 = popt_max[2]
sig_A_max_2 = pcov_max[2,2]
err_A_max_2 = sqrt(sig_A_max_2)

l_max_2 = popt_max[3]
sig_l_max_2 = pcov_max[3,3]
err_l_max_2 = sqrt(sig_l_max_2)

plt.plot(

```

```

t,
expo(t, *popt_max),
label = 'Überschätzung',
ls = '--',
color = c.WINE_RED
)

plt.plot(
# t,
# expo(t, *popt),
0,
0,
label = 'Regression',
color = c.ORANGE,
)

text_pos_upper_2 = 3.2
plt.text(0, exp(text_pos_upper_2),
'$A_{max,1} = $' + str(pap.round_sig_digs(err_A_max_1, A_max_1)[3]))

plt.text(0, exp(text_pos_upper_2 - 0.15),
'$\\lambda_{max,1} = $' + str(pap.round_sig_digs(err_l_max_1, l_max_1)[3]))

plt.text(0, exp(text_pos_upper_2 - 0.3),
'$A_{max,2} = $' + str(pap.round_sig_digs(err_A_max_2, A_max_2)[3]))

plt.text(0, exp(text_pos_upper_2 - 0.45),
'$\\lambda_{max,2} = $' + str(pap.round_sig_digs(err_l_max_2, l_max_2)[3]))

plt.grid(True, ls='--', alpha=0.6)

plt.legend(loc = 'upper center')

# Kleines Fenster oben rechts
a = plt.axes([.65, .63, .2, .2], facecolor = 'white')

plt.plot(
t,
expo(t, *popt_min),
color = c.LIGHT_GREEN,
ls = 'dotted'
)

plt.plot(
t,
expo(t, *popt_max),
color = c.WINE_RED,
ls = '--'
)

plt.plot(
t,
expo(t, *popt),
color = c.ORANGE,
)

plt.errorbar(
t,
N,
yerr = sig_N,
fmt = '.',
capsize = 3.5,
color = c.BLUE
)

plt.xlim(300, 350)
plt.ylim(15, 35)
plt.title('Visualisierung der Abweichung')

plt.grid(True, ls='--', alpha=0.6)

plt.savefig(f'{plt_save_folder}Regression_der_Messdaten_{aufgabe}.pdf', dpi=300)

plt.show()

```

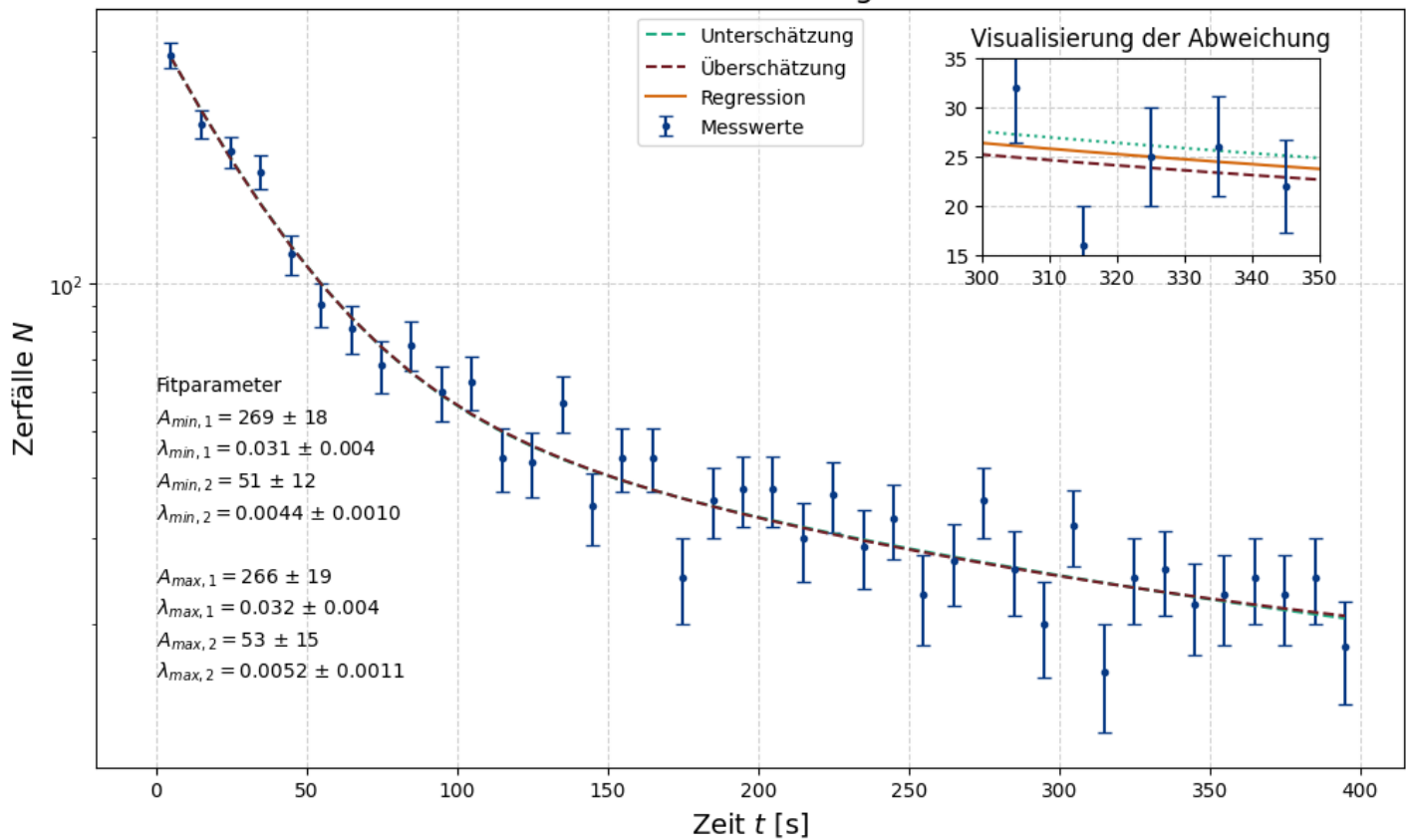
<>:19: SyntaxWarning: invalid escape sequence '\p'

<>:19: SyntaxWarning: invalid escape sequence '\p'

/tmp/ipykernel_17254/199836876.py:19: SyntaxWarning: invalid escape sequence '\p'

plt.title('Zerfälle von Silber mit Untergrund λ_{pm} Fehler', size = 16)

Zerfälle von Silber mit Untergrund $\pm$ Fehler



```
In [ ]: # Differenz der Zerfallskonstanten:
sig_min_l_1 = np.abs(l_1 - l_min_1)
sig_max_l_1 = np.abs(l_1 - l_max_1)

print(f'Min (-) Diff 1: {sig_min_l_1:.2g}\nMax (+) Diff 1: {sig_max_l_1:.2g}')

sig_min_l_2 = np.abs(l_2 - l_min_2)
sig_max_l_2 = np.abs(l_2 - l_max_2)

print(f'Min (-) Diff 2: {sig_min_l_2:.2g}\nMax (+) Diff 2: {sig_max_l_2:.2g}')

print('-'*30)

err_l_1 = sqrt(((sig_min_l_1 + sig_max_l_1) / 2)**2 + sig_l_1**2)
err_l_2 = sqrt(((sig_min_l_2 + sig_max_l_2) / 2)**2 + sig_l_2**2)

print(fr'Zerfallskonstante \lambda_1 = ({pap.round_sig_digs(err_l_1, l_1)[2]}) 1/s')
print(fr'Zerfallskonstante \lambda_2 = ({pap.round_sig_digs(err_l_2, l_2)[2]}) 1/s')

print('-'*30)

# Halbwertszeiten
T_half_1 = log(2) / l_1
err_T_half_1 = log(2) * err_l_1 / (l_1**2)

T_half_2 = log(2) / l_2
err_T_half_2 = log(2) * err_l_2 / (l_2**2)

print(fr'Halbwertszeit T_Halb_1 = ({pap.round_sig_digs(err_T_half_1, T_half_1)[2]}) s')
print(fr'Halbwertszeit T_Halb_2 = ({pap.round_sig_digs(err_T_half_2, T_half_2)[2]}) s')

print('-'*30)

# Lebensdauern
tau_1 = 1 / l_1
err_tau_1 = 1 * err_l_1 / (l_1**2)

tau_2 = 1 / l_2
err_tau_2 = 1 * err_l_2 / (l_2**2)

print(fr'Lebensdauer tau_1 = ({pap.round_sig_digs(err_tau_1, tau_1)[2]}) s')
print(fr'Lebensdauer tau_2 = ({pap.round_sig_digs(err_tau_2, tau_2)[2]}) s')
```

```

Min (-) Diff 1: 0.00027
Max (+) Diff 1: 0.00032
Min (-) Diff 2: 0.00035
Max (+) Diff 2: 0.0004
-----
Zerfallskonstante l_1 = (0.0315 \pm 0.0003) 1/s
Zerfallskonstante l_2 = (0.0048 \pm 0.0004) 1/s
-----
Halbwertszeit T_Halb_1 = (22.04 \pm 0.21) s
Halbwertszeit T_Halb_2 = (145 \pm 11) s
-----
Lebensdauer tau_1 = (31.8 \pm 0.3) s
Lebensdauer tau_2 = (209 \pm 16) s

```

```

In [9]: # Standardabweichung
print(fr'Standardabweichung für ^110 Ag: {pap.std_abw(T_halb_1, 24.6, err_T_halb_1)}')
print(fr'Standardabweichung für ^108 Ag: {pap.std_abw(T_halb_2, 144.6, err_T_halb_2)}')

# Relativer Fehler
print(fr'Relativer Fehler für ^110 Ag: {err_T_halb_1/T_halb_1:.2g}')
print(fr'Relativer Fehler für ^108 Ag: {err_T_halb_2/T_halb_2:.2g}')

```

```

Standardabweichung für ^110 Ag: 12.48
Standardabweichung für ^108 Ag: 0.01
Relativer Fehler für ^110 Ag: 0.0093
Relativer Fehler für ^108 Ag: 0.078

```

Aufgabe 2) Indium

```

In [10]: # Untergrundmessung
unterg_In = np.loadtxt('data/in/untergrund_In.txt', usecols = [1], skiprows=4)

print(np.mean(unterg_In)*12)
print(np.std(unterg_In) / np.sqrt(len(unterg_In)))
print('- '*15)
mean_unterg_In = 12 * np.mean(unterg_In)
sig_mean_unterg_In = np.std(12 * unterg_In) / np.sqrt(len(unterg_In))

print('Der Mittelwert des Untergrundes bei der Indium Messung beträgt:')
print(pap.round_sig_digs(sig_mean_unterg_In, mean_unterg_In)[2])

```

```

35.519999999999996
0.25133244915848013
-----
Der Mittelwert des Untergrundes bei der Indium Messung beträgt:
36 \pm 3

```

```

In [11]: # Neue Regressionsfunktion
y0 = mean_unterg_In
def expo_In(x, A1, l1):
    return A1 * exp(-l1 * x) + y0

```

```

In [12]: aufgabe = '2a'
# Bestimmung der Zerfallskonstante
N_In = np.loadtxt('data/in/indium.txt', usecols = [1], skiprows=4)
sig_N_In = np.sqrt(N_In)
t_In = np.arange(5, 2800, 115)

# Fit
popt_In, pcov_In = curve_fit(expo_In, t_In[1:], N_In[1:], sigma = sig_N_In[1:],
p0 = [600, 0.002])

# Fitparameter
A_In = pop_t_In[0]
sig_A_In = pcov_In[0,0]
err_A_In = sqrt(sig_A_In)

l_In = pop_t_In[1]
sig_l_In = pcov_In[1,1]
err_l_In = sqrt(sig_l_In)

# Plot
plt.figure(figsize = (12,7))

# Messungen
plt.errorbar(
    t_In,
    N_In,
    yerr = sig_N_In,
    fmt = '.',
    capsize = 3.5,
    label = 'Messwerte',
    color = c.BLUE
)

```



```

plt.xlabel('Zeit $t$ [s]', size = 14)
plt.ylabel('Zerfälle $N$ ', size = 14)
plt.title('Zerfälle von Indium mit Untergrund', size = 16)
plt.yscale('log')

# Regression
plt.plot(
    t_In,
    expo_In(t_In, *popt_In),
    label = 'Regression',
    color = c.ORANGE
)

# Text mit den Fitparametern
text_top_pos_in = 6
plt.text(0, exp(text_top_pos_in), 'Fitparameter:')

plt.text(0, exp(text_top_pos_in - 0.05),
         f'$A_1 = $ {pap.round_sig_digs(err_A_In, A_In)[3]}'
        )

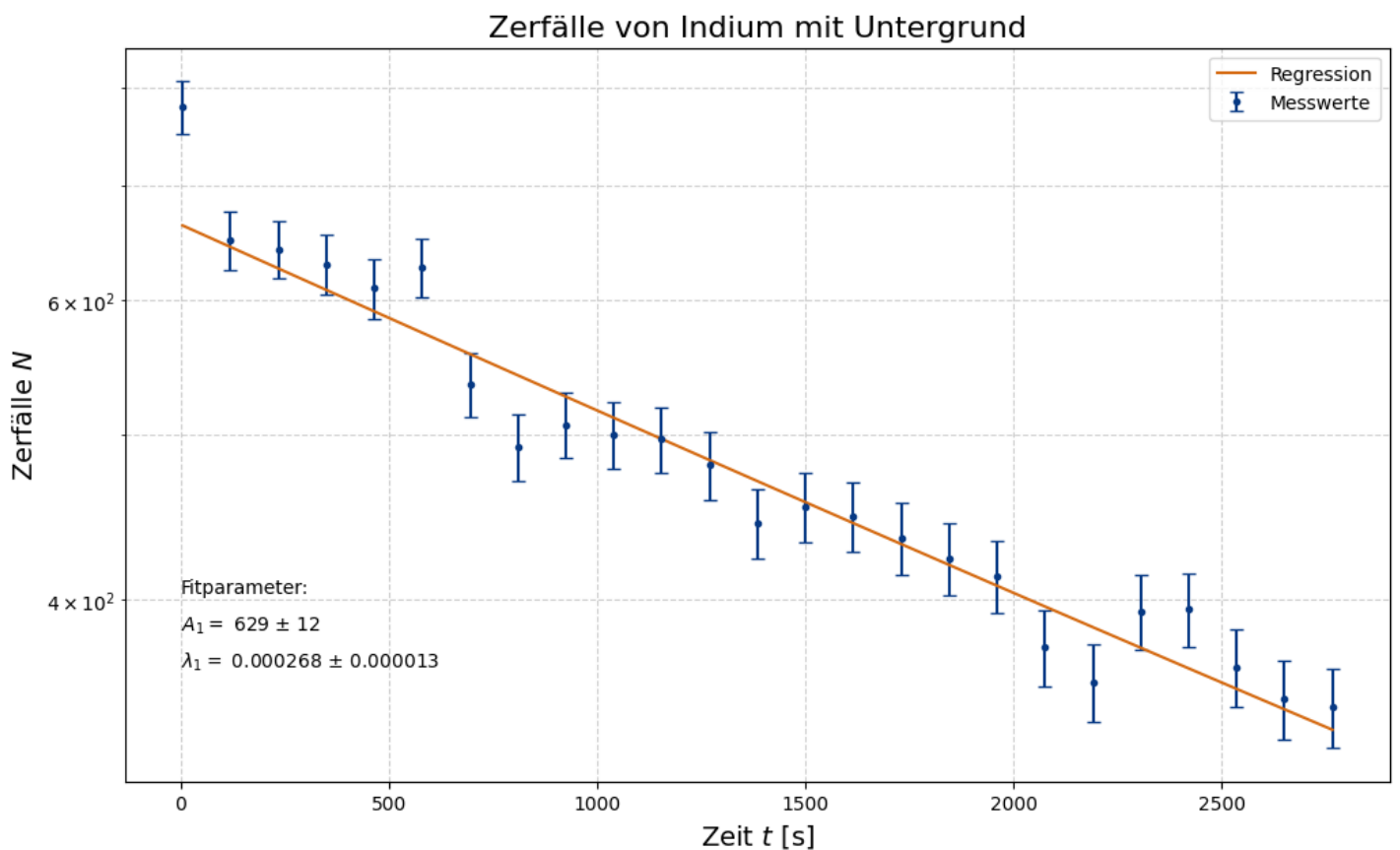
plt.text(0, exp(text_top_pos_in - 0.1),
         f'$\lambda_1 = $ {pap.round_sig_digs(err_l_In, l_In)[3]}'
        )

plt.legend(loc = 'best')
plt.grid(True, 'both', 'both', ls='--', alpha=0.6)

plt.savefig(f'{plt_save_folder}Regression_der_Messdaten_{aufgabe}.pdf', dpi=300)

plt.show()

```



```

In [13]: # Güte des Fits
chi2_ = np.sum((expo_In(t_In, *popt_In) - N_In)**2 / sig_N_In**2)
dof = len(N_In) - 2 #dof:degrees of freedom, Freiheitsgrad
chi2_red = chi2_/dof

print("chi2 =", round(chi2_, 2))
print("chi2_red =", round(chi2_red, 2))

prob = np.round(1 - chi2.cdf(chi2_,dof),2) * 100
print("Wahrscheinlichkeit =", prob, "%")

```

```

chi2 = 39.37
chi2_red = 1.71
Wahrscheinlichkeit = 2.0 %

```

```

In [14]: aufgabe = '2b'
# Fit unter Berücksichtigung des Untergrund-Fehlers
# Fit unterschätzt
y0 = mean_unterg_In - sig_mean_unterg_In
popt_min, pcov_min = curve_fit(expo_In, t_In[1:], N_In[1:], sigma = sig_N_In[1:],

```

```

p0 = [600, 0.002])

# Fitparameter
A_min_In = pop_t_min[0]
sig_A_min_In = pcov_min[0,0]
err_A_min_In = sqrt(sig_A_min_In)

l_min_In = pop_t_min[1]
sig_l_min_In = pcov_min[1,1]
err_l_min_In = sqrt(sig_l_min_In)

# Plot
plt.figure(figsize = (12,7))

# Messungen
plt.errorbar(
    t_In,
    N_In,
    yerr = sig_N_In,
    fmt = '.',
    capsize = 3.5,
    label = 'Messwerte',
    color = c.BLUE
)

plt.xlabel('Zeit  $t$  [s]', size = 14)
plt.ylabel('Zerfälle  $N$ ', size = 14)
plt.title('Zerfälle von Indium mit Untergrund + Fehler', size = 16)
plt.yscale('log')

plt.plot(
    # t_In,
    # expo_In(t_In, *popt_In),
    0,
    0,
    color = c.ORANGE,
    label='Regression'
)

# Plot min
plt.plot(
    t_In,
    expo_In(t_In, *popt_min),
    label = 'Unterschätzung',
    ls = '--',
    color = c.LIGHT_GREEN
)

txt_pos = 6.2
plt.text(0, exp(txt_pos), '$A_{min} = $' + str(pap.round_sig_digs(err_A_min_In, A_min_In)[3]))
plt.text(0, exp(txt_pos - 0.05), '$\lambda_{min} = $' + str(pap.round_sig_digs(err_l_min_In, l_min_In)[3]))

# Fit überschätzt
y0 = mean_unterg_In + sig_mean_unterg_In
popt_max, pcov_max = curve_fit(expo_In, t_In[1:], N_In[1:], sigma = sig_N_In[1:],
p0 = [600, 0.002])

# Fitparameter
A_max_In = pop_t_max[0]
sig_A_max_In = pcov_max[0,0]
err_A_max_In = sqrt(sig_A_max_In)

l_max_In = pop_t_max[1]
sig_l_max_In = pcov_max[1,1]
err_l_max_In = sqrt(sig_l_max_In)

# Plot max
plt.plot(t_In, expo_In(t_In, *popt_max), label = 'Überschätzung', ls = 'dotted',
color = c.WINE_RED)

txt_pos_2 = 6.0
plt.text(0, exp(txt_pos_2), '$A_{max} = $' + str(pap.round_sig_digs(err_A_max_In, A_max_In)[3]))
plt.text(0, exp(txt_pos_2 - 0.05), '$\lambda_{max} = $' + str(pap.round_sig_digs(err_l_max_In, l_max_In)[3]))
plt.legend(loc = 'upper center')
plt.grid(True, 'both', 'both', ls='--', alpha=0.6)

# Kleines Rechteck oben rechts
a = plt.axes([.65, .6, .2, .2], facecolor = 'white')

plt.plot(
    t_In,
    expo_In(t_In, *popt_min),
    color = c.LIGHT_GREEN,
    ls = 'dotted'
)

plt.plot(
    t_In,

```

```

    expo_In(t_In, *popt_max),
    color = c.WINE_RED,
    ls = '--'
)

plt.plot(
    t_In,
    expo_In(t_In, *popt_In),
    color = c.ORANGE
)

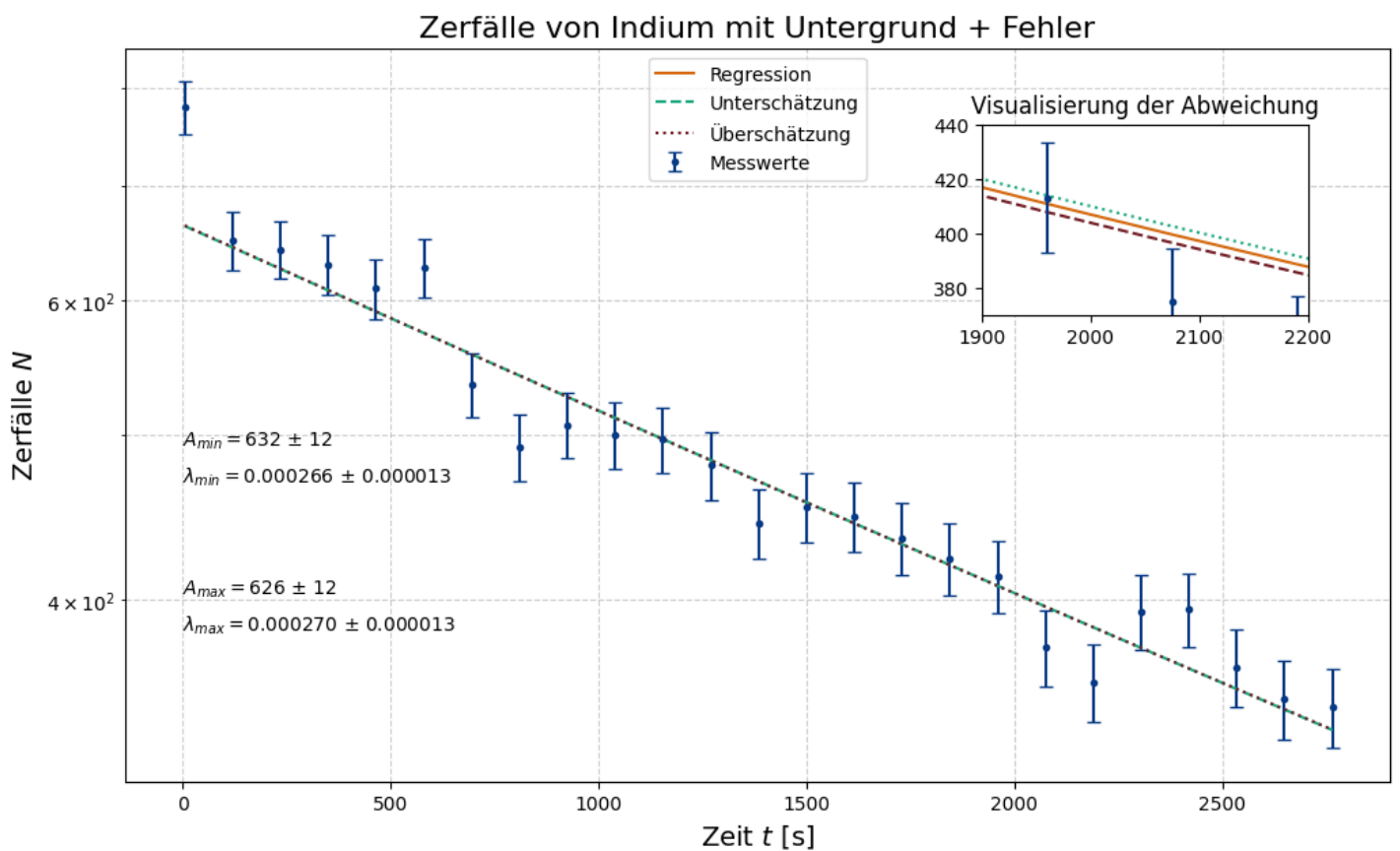
plt.errorbar(
    t_In,
    N_In,
    yerr = sig_N_In,
    fmt = '.',
    capsize = 3.5,
    color = c.BLUE
)

plt.xlim(1900, 2200)
plt.ylim(370, 440)
plt.title('Visualisierung der Abweichung')

plt.savefig(f'{plt_save_folder}Regression_der_Messdaten_{aufgabe}.pdf', dpi=300)

plt.show()

```



```

In [ ]: # Differenz der Zerfallskonstanten:
sig_min_l_In = np.abs(l_In - l_min_In)
sig_max_l_In = np.abs(l_In - l_max_In)

print(f'Min (-): {sig_min_l_In:.3g}\nMax (+){sig_max_l_In:.3g}')

print('-.*30)

err_l_In = sqrt( ((sig_min_l_In + sig_max_l_In) / 2)**2 + sig_l_In**2 )

pot = 1e-4
print(fr'Zerfallskonstante l_In = ({pap.round_sig_digs(err_l_In/pot, l_In/pot)[2]}) * {pot:.1g} 1/s')

T_half_In = log(2) / l_In
err_T_half_In = log(2) * err_l_In / (l_In**2)
print(fr'Halbwertszeit T_half = ({pap.round_sig_digs(err_T_half_In, T_half_In)[2]})s')

tau_In = 1/l_In
err_tau_In = 1 * err_l_In / (l_In**2)
print(fr'Lebensdauer tau = ({pap.round_sig_digs(err_tau_In, tau_In)[2]})s')

```

Min (-): 1.9e-06

Max (+)1.93e-06

1.7362350399580834e-10

Zerfallskonstante $\lambda_{\text{In}} = (2.677 \pm 0.019) \cdot 0.0001 \text{ 1/s}$

Halbwertszeit $T_{\text{halb}} = (2589 \pm 19) \text{ s}$

Lebensdauer $\tau = (3730 \pm 30) \text{ s}$

```
In [16]: # Standardabweichung
print(fr'Standardabweichung für Indum: {pap.std_abw(T_half_In, 3240, err_T_half_In, 1)}')

# Relativer Fehler
print(fr'Relativer Fehler für Indium: {err_T_half_In/T_half_In:.2g}')
```

Standardabweichung für Indum: 35.13

Relativer Fehler für Indium: 0.0071

Abbildungsverzeichnis

1.I Aktivierung der natürlichvorkommenden Silberisotope 51 % ^{107}Ag und 49 % ^{109}Ag . [Wag26b]	2
1.II Ausschnitt der Nuklidkarte mit den Halbwertszeiten und der Energie der emittierten Strahlung[Wag26b]	3
1.III Ausschnitt Nuklidkarte nach NuDat (https://www.nndc.bnl.gov/nudat3/)	3
4.I Bestimmte Regressionen für die Silbermessung.	8
4.II Bestimmte Regressionen mit Unter- und Überschätzung für die Silbermessung.	9
4.III Bestimmte Regressionen für die Indiummessung.	10
4.IV Bestimmte Regressionen mit Unter- und Überschätzung für die Indiummessung.	10

Tabellenverzeichnis

4.I	Messergebnisse der Silbermessung.	9
4.II	Messergebnisse der Indiummessung.	10
4.III	Ergebnisse der Güte der Regressionen für Indium und Silber.	11
5.I	Messergebnisse der Indiummessung mit Standardabweichung (4.3) zum Literaturwert nach Abbildung 1.II.	12

Literaturverzeichnis

- [Dud26] Duden. Duden rechtschreibprüfer, 2026.
- [Fin26a] Finn Zeumer. Pap 2, 2026. Zugriff am 17. Februar 2026.
- [Fin26b] Finn Zeumer. Selbsterstellte python-bibliothek "papulator", 2026.
- [Pro26] Proton. Lumo a.i., 2026.
- [Wag25] Dr. J. Wagner. *Physikalisches Praktikum PAP 1 für Studierende der Physik*, pages 4–28. Universität Heidelberg, 2025.
- [Wag26a] Dr. J. Wagner. *Physikalisches Anfängerpraktikum PAP 2.1 für Studierende der Physik*, chapter 252. Universität Heidelberg, 2026.
- [Wag26b] Dr. J. Wagner. *Physikalisches Anfängerpraktikum PAP 2.2 für Studierende der Physik*, chapter 252. Universität Heidelberg, 2026.